

V₁: Unifying Generation and Self-Verification for Parallel Reasoners

Anonymous Authors¹

Abstract

Test-time scaling for complex reasoning tasks shows that leveraging inference-time compute, for example by independently sampling and aggregating multiple solutions, results in significantly better task outcomes. However, a critical bottleneck is *verification*: sampling is only effective if correct solutions can be reliably identified among candidates. While existing approaches typically evaluate candidates independently via scalar scoring, we demonstrate that models are substantially stronger at **pairwise self-verification**. Leveraging this insight, we introduce **V₁**, a framework that unifies generation and verification through efficient pairwise ranking. **V₁** comprises two components: **V₁-Infer**, an uncertainty-guided algorithm using a tournament-based ranking that dynamically allocates self-verification compute to candidate pairs whose relative correctness is most uncertain; and **V₁-PairRL**, an RL framework that **jointly trains** a single model as both generator and pairwise self-verifier, ensuring the verifier adapts to the generator’s evolving distribution. On code generation (LiveCodeBench, CodeContests) and math reasoning (AIME, HMMT) benchmarks, **V₁-Infer** improves Pass@1 by up to 10% over pointwise verification and outperforms recent test-time scaling methods while being significantly more efficient. Furthermore, **V₁-PairRL** achieves 7–9% test-time scaling gains over standard RL and pointwise joint training, and improves base Pass@1 by up to 8.7% over standard RL.

1. Introduction

Large language models (LLMs) have demonstrated remarkable problem-solving abilities, largely driven by the paradigm of “System 2” thinking (OpenAI, 2024a;b;

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

DeepSeek-AI, 2025) of generating extended chains of thought to reflect, refine, and verify answers at inference time. *Parallel reasoning*, which complements this sequential “deep thinking” by sampling multiple independent chains of thought to explore diverse solution paths, has emerged as a powerful technique for test-time scaling (Wang et al., 2023; Cobbe et al., 2021; Pan et al., 2025; Lian et al., 2025; Snell et al., 2024). In this setup, parallel sampling of multiple chains-of-thought is followed by an aggregation step to select the final answer. The simplest form of aggregation is to select the most common solution among the set of candidates (majority voting). While this suffices for domains like math where answers are objective and easily verifiable (Wang et al., 2023), this can not be used for more general domains which do not admit objective ground truth answers. Instead, the ability to accurately *self-verify* solutions can support an aggregation method that selects the correct solution from the candidate set, as long as it exists in the set, even if it isn’t the most common solution. Thus, taking full advantage of parallel reasoning fundamentally hinges on *accurate self-verification*: sampling N solutions is useful if the model can reliably identify the correct one.

Our experiments identify a critical bottleneck in existing approaches that use automatic verification to take advantage of inference-time compute: without a globally comparable scale of solution quality, existing models are not calibrated to evaluate candidate solutions independent of one another. In addition, existing work suggests that in such settings, models used as verifiers are biased towards positively evaluating their own samples, even if those samples are incorrect (Lu et al., 2025). We find that, instead, self-verifying between candidate solutions via pairwise comparison leads to more robust and accurate outcomes. We explore two core questions. First, *how can we enable LLMs to more accurately self-verify candidate solutions obtained via parallel reasoning, by leveraging pairwise candidate self-verification?* Second, given that self-verification is typically applied after a model has already been trained, *can we instead train models to be better at self-verification for parallel reasoning?*

While pairwise ranking has been extensively studied in reward modeling for LLM alignment (Christiano et al., 2017; Ziegler et al., 2019), this approach remains underexplored for self-verification in a parallel reasoning setting. Addi-

tionally, while reinforcement learning is commonly used to improve the solution-generation capabilities of LLMs on verifiable domains such as code and math (Shao et al., 2024; DeepSeek-AI, 2025), no existing methods effectively utilizes the parallel reasoning chains of LLMs at training time to jointly optimize both the generation and self-verification capabilities, resulting in distribution shifts at inference time. We demonstrate that inducing pairwise self-verification capabilities during RL training is an effective technique to improve test-time scaling performance for parallel reasoners. We leverage our observations to develop V₁, a unified framework that includes a strong inference time scaling algorithm for parallel reasoning, as well as an reinforcement learning framework that induces self-verification capabilities during RL training of LLMs with verifiable rewards.

Our contributions include the following:

1. We show that in parallelized reasoning, independent self-verification of candidate solutions suffers from calibration collapse due to lack of comparative reference, while self-aggregation methods like recursive self-aggregation (RSA; Venkatraman et al., 2025) induce diversity collapse where Pass@N monotonically decreases with aggregation steps. This motivates pairwise verification as a principled alternative for self-verification and test-time scaling.

2. We develop V₁-Infer, an uncertainty-guided pairwise verification algorithm. Rather than scoring solutions in isolation, it pairs candidates by employing a Swiss-system tournament refinement strategy that dynamically allocates verification compute to the most uncertain pairs. This approach provides a significant boost in selection accuracy, effectively improving the performance of the model closer to Pass@N of the original sampled responses. Notably, V₁-Infer outperforms or matches RSA while requiring significantly fewer verification calls.

3. We develop V₁-PairRL, an RL framework that co-trains a single model as both generator and pairwise self-verifier. Unlike prior co-training approaches that rely on pointwise rewards (Sareen et al., 2025; Liu et al., 2025a) or offline data (Venkatraman et al., 2025), V₁-PairRL uses an online, co-evolving objective where generation and pairwise verification improve together. This ensures that as the generator improves, the verifier trains on in-distribution data from the model’s current capabilities, thereby leading to stronger self-verification capabilities at inference time.

We evaluate our framework on code generation (LiveCodeBench, CodeContests) and math reasoning (AIME, HMMT) benchmarks. V₁-Infer improves Pass@1 by up to 10% over pointwise verification and matches or exceeds RSA with a fraction of the compute. V₁-PairRL achieves 7–9% test-time scaling gains over pointwise co-training standard RL, and improves base Pass@1 by up to 8.7% over the

standard RL. Our results demonstrate that unified training for solution generation and self-verification capability, coupled with our pairwise self-verification technique, enables effective parallel reasoning.

2. Related Work

Parallel reasoning and test-time scaling. Test-time scaling typically follows two paradigms: sequential refinement (Madaan et al., 2023) or parallel generation of multiple reasoning paths (Cobbe et al., 2021; Setlur et al., 2025; Pan et al., 2025). While parallel scaling allows for the exploration of diverse solutions, it necessitates a robust mechanism to verify and select the correct output. Existing approaches often rely on access to ground-truth verification signals during inference. For instance, mathematical reasoning often leverages majority voting based on exact answer matching (Wang et al., 2023; Snell et al., 2024), while code generation methods rely on executable test cases or execution feedback (Li et al., 2025a; Jain et al., 2025). In contrast, we focus on *self-verification*, where the model must judge the quality of its own parallel generations without access to external feedback or ground-truth oracles.

Self-verification and self-aggregation. Early work demonstrated that LLMs can verify their own Chain-of-Thought (CoT) reasoning (Weng et al., 2023), though recent studies indicate that pointwise self-verification suffers from a bias toward accepting incorrect solutions (Lu et al., 2025). While sequential self-refinement (Madaan et al., 2023; Stechly et al., 2025) explores verification, it does not address the parallel reasoning setting. Alternatively, self-aggregation methods combine solutions from the same model (Venkatraman et al., 2025; Madaan et al., 2025) but often suffer from diversity collapse, leading to the loss of correct solutions. Our work addresses these limitations by adopting pairwise self-verification, which mitigates pointwise bias and preserves solution diversity better than aggregation-based approaches.

Generative verifiers and Co-training. Generative reward models, which produce reasoning before scoring, have been shown to outperform discriminative approaches (Mahan et al., 2024; Zhang et al., 2025), with pairwise ranking often proving more effective than absolute scoring (Jiang et al., 2023; Toshniwal et al., 2025). In this context, Zhao et al. (2025b) propose AggLM, which explicitly trains an aggregator via RL to synthesize correct answers for improving majority voting. However, these approaches typically rely on *separate* verifier or aggregator models, incurring significant overhead in compute, memory, and data curation. Recent co-training methods attempt to unify generation and verification to mitigate this cost (Sareen et al., 2025; Liu et al., 2025a; Wang et al., 2025), but they predominantly rely on

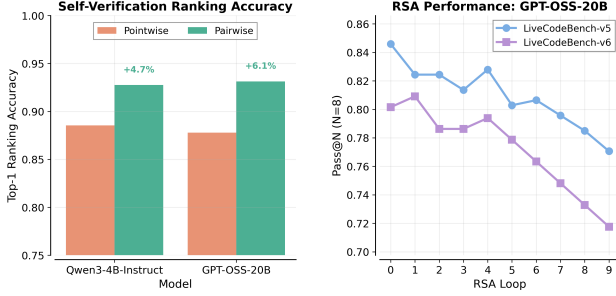


Figure 1. (Left) Pairwise self-verification (using V₁-Infer, §4) outperforms pointwise self-verification in self-verification measured on problems which have both correct and incorrect solutions in their parallel generations (Results with GPT-OSS-20B on LiveCodeBench-V6 prompts). (Right) Recursive self-aggregation of GPT-OSS-20B on LCB-v5 and v6 shows declining Pass@N (diversity collapse). See §3 for more details.

pointwise verification rewards. Our V₁-PairRL framework advances this by co-training a single model for *pairwise* self-verification, eliminating the need for external verifiers while enabling efficient online learning.

See Appendix §A for more discussion about prior work and extension to this section.

3. Why Improve Self-Verification?

From a test-time scaling perspective without external verifiers, parallel reasoning offers two prevalent mechanisms to generate the final solution: **a) self-selection** (verifying and choosing the best candidate) and **b) self-aggregation** (combining solutions to generate a better one). We analyze the limitations of current approaches in both categories to motivate our pairwise framework¹.

Pointwise self-verification suffers from calibration collapse. Standard pointwise verification assigns scalar scores to solutions in isolation. This approach is fundamentally limited by the lack of a comparative reference set. Statistically, latent utilities in choice models (e.g., Bradley–Terry) are identifiable only up to monotonic transformations, meaning absolute scores lack a globally comparable scale (Bradley & Terry, 1952). Consequently, pointwise scores exhibit high variance and poor cross-context calibration, often over-scoring plausible but incorrect solutions. Christiano et al. (2017) noted that for learning from human preferences, relative scores are much easier to provide for humans compared to absolute scores. *Pairwise* judgments simplify the task to a well-posed relative comparison. As shown in Figure 1 (left), this shift to relative ranking yields significantly higher top-1 self-ranking accuracy.

¹Majority voting is less general and only applicable to scenarios with objective ground truth answers, such as math.

Self-aggregation induces diversity collapse. While self-aggregation methods, including Recursive Self-Aggregation (RSA) (Venkatraman et al., 2025; Khairi et al., 2025; Li et al., 2025b; Madaan et al., 2025) can help consolidate parallel reasoning chains and improve Pass@1, they may lead to *diversity collapse*. As shown in Figure 1 (Left), the Pass@N score, representing the probability that *at least one* correct solution exists in a set of generated N solutions, monotonically decreases as aggregation steps increase for RSA. This indicates that RSA frequently discards or degrades correct outlier solutions during refinement. Specifically, since the refined Pass@1 rarely exceeds the *initial* Pass@N of the raw samples, the value of self-aggregation is unclear compared with a strong self-verifier that can select the best answer and reach near Pass@N performance. Instead of relying on implicit self-verification capabilities of aggregation based method, explicit and accurate self-verification can provide orthogonal improvements to aggregator-based approaches since each aggregation step (that may induce diversity collapse) can benefit from self-verification (that maintains pass@N), which can help select promising candidate solutions to aggregate.

Algorithm 1 Uncertainty-Guided Pairwise Ranking

Require: Problem x , candidates $\mathcal{S} = \{s_i\}_{i=1}^N$, pairwise budget B , min-degree $d_{\min}$, Swiss window size h , weight floor τ

Ensure: Ranking π

- 1: **Initialize:** $\forall i$, scores $\mu_i \leftarrow 0.5$ and degrees $d_i \leftarrow 0$
- 2: history $\mathcal{H} \leftarrow \emptyset$ $\triangleright (i, j) \in \mathcal{H}$ iff (s_i, s_j) has been compared
- 3: state $\mathcal{T} \leftarrow (\{\mu_i\}_{i=1}^N, \{d_i\}_{i=1}^N, \mathcal{H})$
- 4: used $\leftarrow 0$
- Phase 1: Topology Coverage**
- 5: **while** used $< B$ **and** $\exists i : d_i < d_{\min}$ **do**
- 6: $\mathcal{P} \leftarrow \text{COVERAGEPAIRS}(\mathcal{T}, d_{\min})$
- 7: $\mathcal{O} \leftarrow \text{PAIRSELFVERIFY}(x, \mathcal{S}, \mathcal{P})$ $\triangleright$ parallel LLM judging
- 8: $\text{UPDATESTATS}(\mathcal{T}, \mathcal{O}, \tau)$ $\triangleright$ update μ_i, d_i ; record $\mathcal{H}$
- 9: used $\leftarrow$ used $+ |\mathcal{P}|$
- 10: **end while**
- Phase 2: Swiss Refinement**
- 11: **while** used $< B$ **and** $N > 2$ **do**
- 12: $\pi \leftarrow \text{RANK}(\{\mu_i\}_{i=1}^N)$ $\triangleright$ descending μ
- 13: $\mathcal{P} \leftarrow \text{SWISSPAIRS}(\pi, \mathcal{T}, h)$ $\triangleright$ within window h ; prefer unseen and near-ties
- 14: $\mathcal{O} \leftarrow \text{PAIRSELFVERIFY}(x, \mathcal{S}, \mathcal{P})$
- 15: $\text{UPDATESTATS}(\mathcal{T}, \mathcal{O}, \tau)$
- 16: used $\leftarrow$ used $+ |\mathcal{P}|$
- 17: **end while**
- 18: **return** $\text{RANK}(\{\mu_i\}_{i=1}^N)$

4. LLMs can Self-Verify with V₁-Infer

Motivated by the limitations of pointwise verification (its inability to capture relative nuances and lack of calibration) and for maintaining solution diversity, we propose V₁-Infer, a pairwise verification framework designed for par-

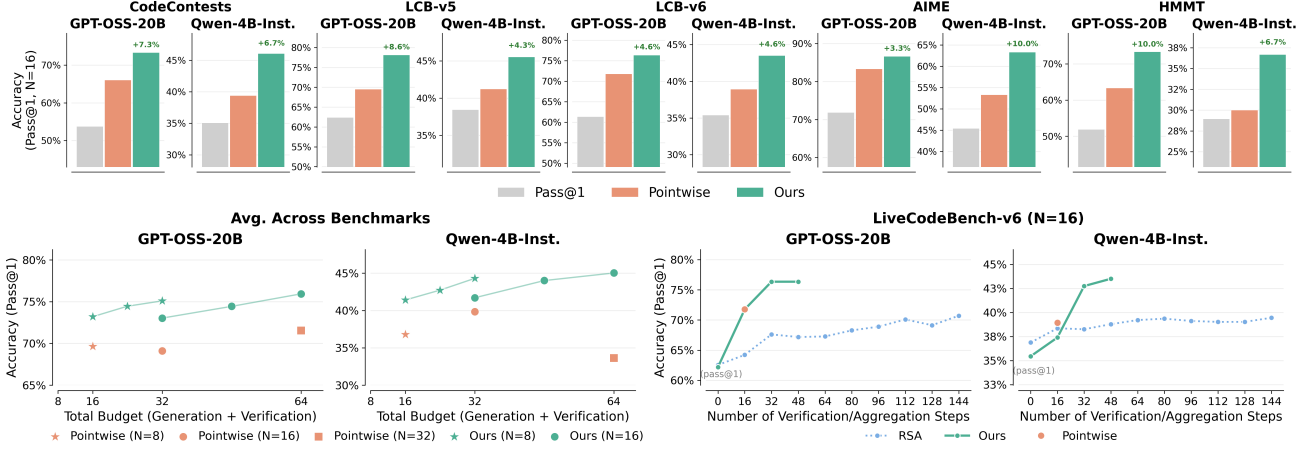


Figure 2. (Top) Performance after self-verification using V₁-Infer compared with pointwise self-verification across benchmarks and models at N=16 base generations. Results presented for GPT-OSS-20B and Qwen-4B-Instruct-2507. (Bottom Left) Accuracy vs. total budget (generation + verification calls). Stars denote $N = 8$ base generations, circles denote $N = 16$. V₁-Infer consistently outperforms pointwise self-verification at equivalent budgets. See Fig. 10 for per benchmark results. (Bottom Right) Comparison with Recursive Self-Aggregation (RSA) (Venkatraman et al., 2025) on LCB-v6. V₁-Infer method achieves higher accuracy with fewer self-verification calls. Results for GPT-OSS-120B and Qwen-4B-Thinking-2507 are in Fig. 7 and show similar trends.

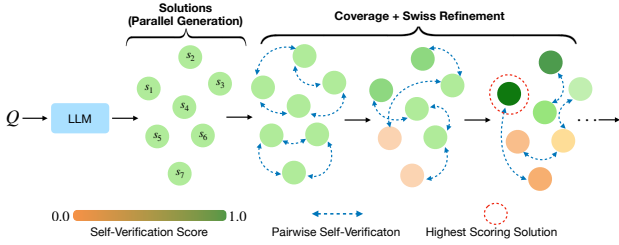


Figure 3. Swiss Refinement Overview. Increasing pairwise verifications enables LLMs to better self-verify for selecting the best response among N self-generated solutions. See Section 4

allel reasoning. While we can use pairwise self-verification to score responses by the model, naively pairing solutions may require quadratic number of $C(N, 2)$ self-verification attempts. Our approach, detailed in Algorithm 1, consists of a weighted aggregation mechanism and a two-phase budgeting strategy designed to maximize information gain with each new pair which allows us to efficiently scale test-time self-verification compute while achieving significant improvement in reasoning performance. A high-level overview is presented in Fig. 3

Uncertainty-Guided Score Aggregation. Standard pairwise voting often treats all “wins” equally, failing to distinguish between a marginal preference (e.g., a 6 vs. 5 rating) and a decisive victory. Instead, we prompt the models to score their solutions in a pairwise setting instead of simply providing “correct” or “incorrect” as the output as in standard pairwise voting, which provides us with fine-grained information about the goodness of solutions. To capture this nuance of relative quality, we adopt a weighted aggrega-

tion scheme in which the magnitude of the rating difference serves as a proxy for the judge’s confidence. Given N candidate solutions S generated by an LLM, and a comparison budget B (number of self-verification LLM calls), a pairwise comparison between (s_i, s_j) using the same LLM outputs calibrated ratings $(r_i, r_j) \in [1, 10]$ for the two solutions. We define a confidence weight

$$w_{ij} = \max\left(\frac{|r_i - r_j|}{9}, \tau\right),$$

where $\tau > 0$ is a small floor that ensures non-zero weights even for near-ties. Let $v_{ij} \in \{0, 0.5, 1\}$ denote the comparison outcome for s_i , corresponding to a win, tie, or loss, respectively. The estimated quality score μ_i is then computed as the uncertainty-weighted win rate

$$\mu_i = \frac{\sum_{j \in \mathcal{N}(i)} w_{ij} v_{ij}}{\sum_{j \in \mathcal{N}(i)} w_{ij}}, \quad (1)$$

where $\mathcal{N}(i)$ denotes the set of opponents compared against s_i . This formulation ensures that high-confidence judgments dominate the global ranking, while ambiguous comparisons contribute minimally to score variance.

Phase 1: Topology Coverage. A primary failure mode in low-budget pairwise ranking is *path dependence*, where solutions become “orphaned” or misranked due to insufficient pairwise comparisons with other solutions. We mitigate this by enforcing a minimum degree constraint to ensure all solutions are pairwise self-verified at least a minimum number of times. We begin with random disjoint pairings to guarantee global connectivity ($d_i \geq 1$), ensuring every solution enters the tournament. Subsequently, we iteratively

target under-sampled nodes ($d_i < d_{\min}$) to meet a minimum degree threshold. Crucially, rather than selecting random opponents, we pair these low-degree nodes with candidates having the closest current mean score μ . This “anchors” solutions against comparable peers early in the process, preventing initial noise from propagating into the refinement phase.

Phase 2: Swiss Refinement. With the topology anchored, the remaining budget focuses on resolving rank ambiguity through an uncertainty-aware Swiss system. In each round, solutions are sorted by their current score μ , and we pair neighbors within a local window to minimize the score gap $|\mu_i - \mu_j|$ among unseen pairs. This strategy is grounded in active learning principles: under Bradley-Terry models, comparisons between items of similar skill (near-ties) yield the highest marginal information gain. By concentrating the judge budget on these ambiguous decision boundaries, V₁-Infer efficiently reduces uncertainty where it matters most, achieving high ranking accuracy even with a sparse comparison graph ($K \ll N$). See Appendix §D for the detailed algorithm and full specification of the helper procedures.

4.1. Experimental Settings

Models and Benchmarks. We evaluate four diverse models: GPT-OSS-20B, Qwen3-4B-Instruct, GPT-OSS-120B, and Qwen-4B-Thinking. For code generation, we use LiveCodeBench-v5, LiveCodeBench-v6 (Jain et al., 2024), and CodeContests (Li et al., 2022). For math, we use AIME’25 and HMMT’25 (Balunović et al., 2025).

Evaluation Protocol. For all methods, we first generate N candidate solutions independently from the model, then apply the verification strategy to select the final answer. We use $N \in \{8, 16\}$ for most experiments, and additionally $N = 32$ for pointwise verification in budget-matched comparisons. Detailed parameters for inference sampling and swiss-refinement algorithm are provided in Appendix C.

Pointwise Baseline. For fair comparison, pointwise self-verification uses the same 1–10 grading system as our pairwise method. The model evaluates each solution in isolation by assigning a score between 1 and 10, and the solution with the highest score is selected. This ensures that any performance differences stem from the pairwise comparison structure rather than the scoring mechanism. The exact prompts for both pointwise and pairwise verification (for code and math) are provided in Appendix E.

Verification Budget. V₁-Infer allows flexible control over the verification budget. We report results for budget multipliers of 1 $\times$, 2 $\times$, and 3 $\times$ the number of initially generated solutions (N). For example, with $N = 16$ and budget 2 $\times$, we perform 32 pairwise comparisons. Figure 2 provides an overview of our results, comparing pointwise verification

with pairwise verification (top), budget-matched evaluation (bottom left), and budgeted comparison with Recursive Self-Aggregation (bottom right).

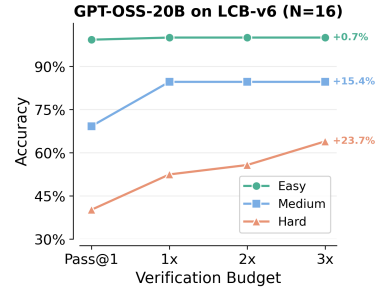


Figure 4. Accuracy improvement with increasing verification budget across problem difficulty levels (GPT-OSS-20B on LiveCodeBench-v6, N=16). V₁-Infer provides the largest gains on hard problems (+23.7%), where baseline Pass@1 is lowest and selection among parallel samples is most valuable.

4.2. Results

Our goal is to measure the efficacy of V₁-Infer compared to standard LLM-as-a-judge (pointwise) verification for parallel reasoners, and analyze how it compares with pointwise and aggregation-based test-time scaling methods in a budget-matched setting. Our results are summarized in Figure 2

V₁-Infer consistently outperforms pointwise self-verification. On CodeContests, GPT-OSS-20B improves from 66.06% to 73.33% (+7.3%), while Qwen3-4B-Instruct improves from 39.4% to 46.1% (+6.7%). On LiveCodeBench-v5, GPT-OSS-20B gains +8.6% and Qwen3-4B-Instruct gains +4.3%. On HMMT, GPT-OSS-20B gains +10.0% and Qwen3-4B-Instruct gains +6.7%. See Figure 2 (top) for more results. These results demonstrate that pairwise comparisons provide more informative signals than independent pointwise scores. We further observe that as verification budget increases, pairwise verification performance improves or stays consistent across most models and benchmarks (see Appendix Figure 7). While the results above are for same number of base solution generations (N=16) we find similar trends while comparing methods in a compute matched setting as well, Figure 2 (bottom left), for e.g., with compute budget of 64 model calls, with Qwen-4B-Instruct-2507, pointwise verification performs approx. 33% while pairwise verification reaches 45% accuracy on average on code gen. benchmarks.

V₁-Infer enables improved test-time scaling. We compare against Recursive Self-Aggregation (RSA) (Venkatraman et al., 2025), a state-of-the-art test-time scaling method that iteratively refines solutions through an evolutionary self-aggregation process. As shown in Figure 2 (bottom right), V₁-Infer achieves higher accuracy with significantly fewer LLM calls. On LiveCodeBench-v6 with N=16, our method reaches 76% Pass@1 with only 48 verification calls,

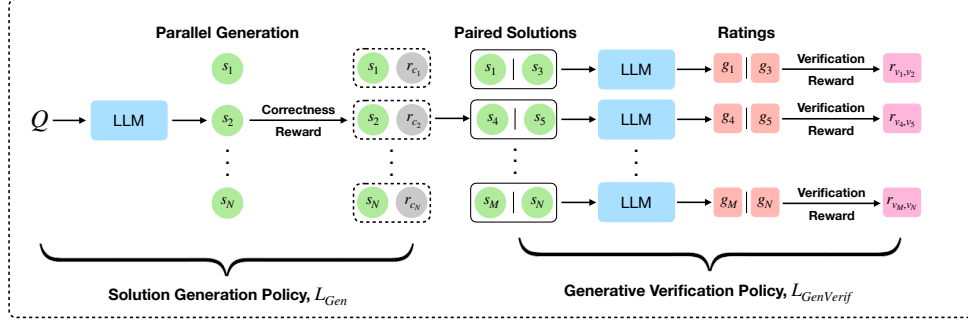


Figure 5. **V₁-PairRL: Unified RL training for co-evolving generation and pairwise verification.** A single LLM is trained with two objectives: J_{Gen} optimizes solution generation using correctness rewards, while $J_{\text{PairVerif}}$ optimizes pairwise verification accuracy. The generator produces G solutions per problem, which are evaluated for correctness and paired for verification training. The verifier evaluates solutions in a paired setting by providing correctness scores for both solutions. Since we are in a verifiable setting and know from ground truth (such as test case execution during code generation) which of the generator solutions are correct, we can calculate correctness rewards for the verifier as well. See Section 5 for more details on how optimization proceeds and rewards are calculated in this framework.

higher compared to the maximum accuracy attained by RSA. This efficiency stems from our uncertainty-guided Swiss refinement, which concentrates comparisons on informative pairs rather than exhaustively aggregating or verifying all solution pairs. See Figures. 8, 9, 10 for extended results.

V₁-Infer provides the largest gains on hard problems.

Figure 4 shows how V₁-Infer improves accuracy across different problem difficulty levels. On easy problems, Pass@1 is already near-optimal (99.3%). However, on hard problems where Pass@1 is only 40.2%, pairwise verification with budget 3x achieves 63.9%, a gain of +23.7%. Medium difficulty problems show an improvement of +15.4%. This pattern demonstrates that V₁-Infer is most valuable precisely where it is needed most: on challenging problems where the model generates a diverse set of candidate solutions and accurate selection among them is critical for bridging the gap between Pass@1 and Pass@N.

V₁-Infer outperforms random pairwise verification. To validate the effectiveness of our uncertainty-guided refinement algorithm, we compare against a baseline that randomly selects pairs for pairwise comparison. On LCB-v6 with GPT-OSS-20B at budget 3x, V₁-Infer achieves 76.3% accuracy compared to 72.5% for random pairing, a gain of +3.8%. This demonstrates that the strategic pair selection in V₁-Infer, which prioritizes comparisons between solutions with similar estimated quality scores, yields more informative judgments than naive random sampling.

Takeaway:

Pairwise self-verification (V₁-Infer) yields more accurate candidate ranking than pointwise scoring and enables improved performance by test-time scaling of verification compute, compared with self-aggregation based test-time scaling.

5. Improving Parallel Self-Verification via Unified RL Training

The previous section demonstrated that pairwise self-verification significantly outperforms pointwise approaches at inference time. A natural question arises: *can we explicitly train models to become stronger self-verifiers?* Current RL paradigms for reasoning focus almost exclusively on optimizing the generation of correct solutions, treating verification as either an afterthought or an external process. While recent work has explored co-training generators with verifiers (Sareen et al., 2025; Liu et al., 2025a), these approaches rely on *pointwise* rewards and fail to leverage the parallel responses that techniques like GRPO naturally generate during training. Additionally, pointwise verification is uncalibrated, which may make optimization hard. Other methods train for aggregation awareness (Venkatraman et al., 2025; Zhao et al., 2025b) but use offline data, limiting the model’s ability to adapt to its own evolving generation distribution during RLVR training.

We propose V₁-PairRL, a unified RL framework that trains a single LLM to be both a strong reasoner and an accurate *pairwise* self-verifier. The key insight is that generation and verification should *co-evolve*: as the generator improves, the distribution of responses changes, and the verifier must learn score increasingly high-quality solutions. This online, co-evolving setup ensures verification training data is always in-distribution for the model’s current capabilities. However, unified training introduces unique challenges: naive implementations suffer from reward hacking, where the generator and verifier collude to maximize reward without improving actual capabilities. We address these challenges through careful reward design and pairing strategies.

5.1. Preliminaries: RL for LLMs

The goal of RL for LLMs is to maximize expected reward r of an LLM policy π_θ over a distribution of prompts $P(Q)$.

This is commonly done with policy gradient methods. Specifically, Group-Relative Policy Optimization (GRPO) (DeepSeek-AI, 2025) and its variants (Yu et al., 2025) have shown significant promise and stable optimization dynamics. In each episode, we sample prompts $q \sim P(Q)$ and a group of G rollouts per prompt $\{o_i\} \sim \pi_{\text{old}}(\cdot|q)$. We maximize $J_{\text{Gen}}(\theta)$:

$$\mathbb{E}_{q, \{o_i\}} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1, t=1}^{G, |o_i|} \min \left(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) A_i \right) \right] \quad (2)$$

where $A_i = r_i - \text{mean}(\mathbf{r})$ is the advantage computed over the group and $\rho_{i,t} = \pi_{\theta}(o_{i,t}|q, o_{i,<t}) / \pi_{\text{old}}(o_{i,t}|q, o_{i,<t})$ is the importance sampling ratio.

5.2. Co-evolving Solver-Verifier Training

Our training objective combines generation and pairwise verification in a unified RL formulation:

$$J(\theta) = J_{\text{Gen}}(\theta) + \lambda J_{\text{PairVerif}}(\theta) \quad (3)$$

J_{Gen} optimizes the likelihood of correct reasoning paths (using GRPO) and $J_{\text{PairVerif}}$ optimizes the accuracy of pairwise ranking judgments. Crucially, both objectives operate on the *same* rollouts: during each training step, the model generates G solutions per problem, which are used both to compute generation rewards and to form solution pairs for verification training. This ensures the verifier always trains on in-distribution data from the current policy. Figure 5 shows an overview of V₁-PairRL. While our framework is general, we instantiate experiments on RL for code-generation following DeepCoder (Luo et al., 2025).

Rewards. For the solution generator, we use a standard binary correctness reward $r_{\text{gen}} \in \{0, 1\}$ based on passing all ground truth test cases. For the pairwise self-verification objective, the model compares two solutions (s_A, s_B) and outputs a confidence score between 1 and 10, which is normalized to $v_i \in [0, 1]$ for each solution. We reward the verifier based on how well its scores align with ground truth correctness $y_i \in \{0, 1\}$:

$$r_{\text{verif}} = \frac{1}{2} \sum_{i \in \{A, B\}} \mathbb{I}(|v_i - y_i| \leq 0.2) \cdot (1 - |v_i - y_i|) \quad (4)$$

The indicator function $\mathbb{I}(|v_i - y_i| \leq 0.2)$ implements a *sparsity threshold*: the verifier receives reward only when its score is within 0.2 of the ground truth (i.e., scoring a correct solution ≥ 0.8 or an incorrect solution ≤ 0.2). This design choice is critical for preventing reward hacking, as we discuss below.

Mitigating Reward Hacking. Unified training of generators and verifiers is prone to specific collapse modes. We address two critical forms of reward hacking:

1. *The Safe Bet Collapse:* Without the **sparsity threshold**, the verifier learns to output a safe, middle-ground score (e.g., $v_i = 0.5$) for every solution. This minimizes the risk of being “very wrong” but yields a meaningless discriminator. Sparsity threshold forces the model to commit to confident judgments: only scores near 0 or 1 receive positive reward.

2. *The Empty Solution Loop:* If the verifier is trained on pairs of two incorrect solutions, the generator may collapse into producing empty or trivially incorrect outputs. The verifier easily identifies these as incorrect (scoring them near 0), receiving high reward. This creates a situation where the generator degrades to maximize the verifier’s ease of judgment. To prevent this, we enforce a strict pairing strategy: we only trigger verification training when we can form pairs containing at least one correct solution (Correct-Incorrect or Correct-Correct pairs).

5.3. Experimental Settings

We instantiate V₁ on RLVR for code generation following the DeepCoder recipe (Luo et al., 2025). Models are trained on the DeepCoder training set, which comprises 24K verified coding problems, utilizing binary rewards determined by executing generated code against ground-truth test cases (1 if all pass, 0 otherwise). We track solution generation performance on the DeepCoder validation set.

Models and Benchmarks. We train Qwen3-4B-Instruct-2507, an instruction-tuned model, following the DeepCoder experimental protocol. We evaluate trained models on LiveCodeBench V5, V6, and CodeContests. See Appendix §C.1 for evaluation hyperparameters.

Pairwise Verification Training. For $J_{\text{PairVerif}}$, we group multiple verification prompts (problem + solution pairs from the solver) rather than generating multiple rollouts for a single prompt. This strategy allows us to leverage information from all pairwise comparisons without increasing the total rollout budget. The advantage for verification rollouts reduces to a REINFORCE-style estimator with a mean baseline calculated across prompts.

Baselines. We compare our approach against two primary baselines: (1) a standard **RL baseline** trained solely for generation without a verification objective, and (2) **V₁-PointRL**, a model jointly trained with pointwise verification rewards under the same setup. Training details for the pointwise baseline are provided in Appendix B.

Training Setup. We adopt the DAPO (Yu et al., 2025) configuration, removing the KL penalty, using Clip High, and applying token-level loss, and follow Dr. GRPO (Liu et al., 2025b) by removing standard deviation normalization. To ensure a fair comparison, we enforce a fixed compute budget of 8 total rollouts per problem. The baseline allocates all 8 rollouts to the solver, while co-evolving models (V₁-PairRL

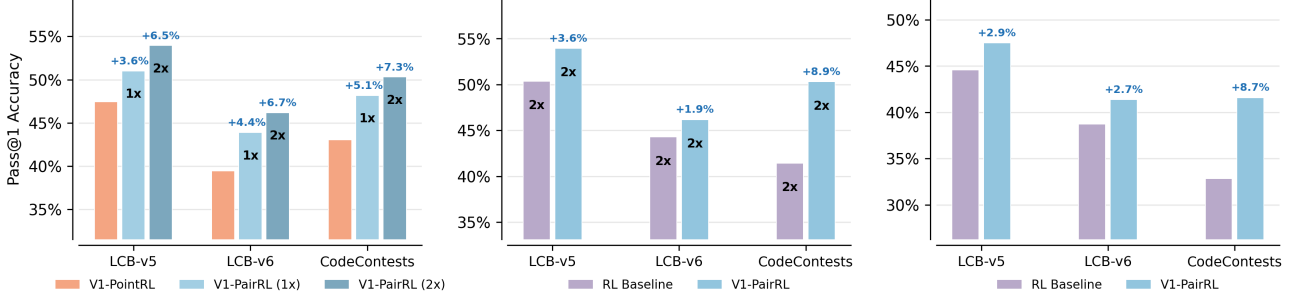


Figure 6. **V₁-PairRL training results (N=16).** (left) Test-time scaling: V₁-PairRL outperforms V₁-PointRL and improves with increased verification budget. (middle) With V₁-Infer at 2x budget: V₁-PairRL outperforms RL baseline even when both use pairwise verification. (right) Base Pass@1: Co-training with pairwise verification improves generation quality over the RL baseline.

and PointRL) split this budget into 4 solver and 4 verifier rollouts. We verified that training the baseline for a longer duration with fewer rollouts does not improve performance, confirming that gains stem from the co-training objective. All models are trained for 150 steps, with checkpoints selected based on the validation accuracy (pass@1).

5.4. Results

V₁-PairRL Offers Superior Test-Time Scaling Capabilities. To evaluate the test-time scaling benefits of co-trained verification, we compare V₁-PairRL against V₁-PointRL, our pointwise verification baseline trained with the same co-evolving setup. As shown in Figure 6(a), pairwise self-verification consistently outperforms pointwise across all benchmarks at N=16. On LiveCodeBench-v5, V₁-PairRL achieves 53.9% (2x budget) compared to 47.4% for V₁-PointRL (+6.5%). Similar improvements are observed on LiveCodeBench-v6 (+6.8%) and CodeContests (+7.3%). V₁-PairRL exhibits positive scaling with verification budget: increasing budget yields consistent accuracy gains across all benchmarks, demonstrating that the model learns to effectively leverage additional pairwise comparisons.

V₁-PairRL with V₁-Infer Outperforms RL Baseline with V₁-Infer. A key question is that in a test-time scaling setup with the same algorithm, does V₁-PairRL show improved performance compared to the standard RL baseline. To investigate, we apply V₁-Infer at inference time to both V₁-PairRL and the RL baseline, using identical 2x verification budgets. As shown in Figure 6 (middle), V₁-PairRL consistently outperforms the RL baseline even when both leverage pairwise verification: +3.6% on LiveCodeBench-v5, +1.9% on LiveCodeBench-v6, and +8.9% on CodeContests. This shows that co-training with pairwise verification enhances overall performance in an equivalent test-time scaling setup.

V₁-PairRL improves RL Baseline. Co-training for pairwise self-verification yields substantial improvements in generation quality. As shown in Figure 6(right), V₁-PairRL achieves consistent Pass@1 improvements over the RL baseline across all three code generation benchmarks at N=16:

+2.9% on LiveCodeBench-v5, +2.7% on LiveCodeBench-v6, and +8.7% on CodeContests. The gains demonstrate that jointly optimizing for generation and pairwise verification creates a beneficial learning signal that improves the model’s underlying reasoning capabilities (which can also include improving the self-verification capability within long chains of thought), not just its verification accuracy. Our results echo prior findings by Sareen et al. (2025), who showed that co-training with pointwise verification improves the base model’s Pass@1 performance. We show that pairwise verification takes this one step further: V₁-PairRL also outperforms V₁-PointRL in generation quality, particularly on CodeContests (by about +6%), where the gap is most pronounced (see Appendix Figure 11).

Takeaway:

Unified RL training that jointly optimizes generation and pairwise verification (V₁-PairRL) produces stronger reasoning models and improves test-time scaling compared with generation-only baselines or models co-trained with pointwise verification.

6. Conclusion

We presented V₁, a unified framework for advancing self-verification in parallel reasoning, grounded in the insight that pairwise comparison constitutes a fundamentally more robust primitive for verification than absolute scoring. We introduced V₁-Infer, which employs tournament-based refinement to dynamically allocate compute toward ambiguous pairs, significantly outperforming pointwise verification and prior aggregation techniques with lower computational overhead. Crucially, V₁-PairRL establishes that jointly training a single model for both generation and pairwise self-verification unlocks superior test-time scaling compared to standard RL and pointwise baselines. By unifying generation with pairwise verification, V₁ provides a robust framework for both effective RL training and scalable parallel reasoning.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Balunović, M., Dekoninck, J., Petrov, I., Jovanović, N., and Vechev, M. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL <https://matharena.ai/>.
- Bradley, R. and Terry, M. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Ikmd3fKBPQ>.
- Jain, A. K., Gonzalez-Pumariaga, G., Chen, W., Rush, A. M., Zhao, W., and Choudhury, S. Multi-turn code generation through single-step rewards. In *International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=aJeLhLcsh0>.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Jiang, D., Ren, X., and Lin, B. Y. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14165–14178, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.792. URL <https://aclanthology.org/2023.acl-long.792/>.
- Khairi, A., D’souza, D., Fadaee, M., and Kreutzer, J. Making, not taking, the best of n, 2025. URL <https://arxiv.org/abs/2510.00931>.
- Lee, D., Hwang, C., and Lee, K. Learning to generate unit test via adversarial reinforcement learning, 2025. URL <https://arxiv.org/abs/2508.21107>.
- Li, D., Cao, S., Cao, C., Li, X., Tan, S., Keutzer, K., Xing, J., Gonzalez, J. E., and Stoica, I. S*: Test time scaling for code generation, 2025a. URL <https://arxiv.org/abs/2502.14382>.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., Hubert, T., Choy, P., de Masson d’Autume, C., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Goyal, S., Cherepanov, A., Molloy, J., Mankowitz, D. J., Robson, E. S., Kohli, P., de Freitas, N., Kavukcuoglu, K., and Vinyals, O. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.
- Li, Z., Feng, X., Cai, Y., Zhang, Z., Liu, T., Liang, C., Chen, W., Wang, H., and Zhao, T. Llm can generate a better answer by aggregating their own responses. *arXiv preprint arXiv:2503.04104*, 2025b.
- Lian, L., Wang, S., Juefei-Xu, F., Fu, T.-J., Li, X., Yala, A., Darrell, T., Suhr, A., Tian, Y., and Lin, X. V. Threadweaver: Adaptive threading for efficient parallel reasoning in language models, 2025. URL <https://arxiv.org/abs/2512.07843>.
- Liu, X., Liang, T., He, Z., Xu, J., Wang, W., He, P., Tu, Z., Mi, H., and Yu, D. Trust, but verify: A self-verification approach to reinforcement learning with verifiable rewards, 2025a. URL <https://arxiv.org/abs/2505.13445>.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding r1-zero-like training: A critical perspective, 2025b. URL <https://arxiv.org/abs/2503.20783>.
- Liu, Z., Wang, P., Xu, R., Ma, S., Ruan, C., Li, P., Liu, Y., and Wu, Y. Inference-time scaling for generalist reward modeling, 2025c. URL <https://arxiv.org/abs/2504.02495>.
- Lu, J., Teehan, R., Jin, J., and Ren, M. When does verification pay off? a closer look at llms as solution verifiers, 2025. URL <https://arxiv.org/abs/2512.02304>.

- Luo, M., Tan, S., Huang, R., Patel, A., Ariyak, A., Wu, Q., Shi, X., Xin, R., Cai, C., Weber, M., Zhang, C., Li, L. E., Popa, R. A., and Stoica, I. Deepcoder: A fully open-source 14b coder at o3-mini level. *Notion Blog*, 2025.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Madaan, L., Didolkar, A., Gururangan, S., Quan, J., Silva, R., Salakhutdinov, R., Zaheer, M., Arora, S., and Goyal, A. Rethinking thinking tokens: Llm as improvement operators, 2025. URL <https://arxiv.org/abs/2510.01123>.
- Mahan, D., Van Phung, D., Rafailov, R., Blagden, C., Lile, N., Castriato, L., Fränken, J.-P., Finn, C., and Albalak, A. Generative reward models. *arXiv preprint arXiv:2410.12832*, 2024.
- Mahdavi, S., Kisacanin, B., Toshniwal, S., Du, W., Moshkov, I., Armstrong, G., Liao, R., Thrampoulidis, C., and Gitman, I. Scaling generative verifiers for natural language mathematical proof verification and selection. *arXiv preprint arXiv:2511.13027*, 2025.
- OpenAI. Learning to reason with LLMs, 2024a. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- OpenAI. Learning to reason with llms, 2024b. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- Pan, J., Li, X., Lian, L., Snell, C., Zhou, Y., Yala, A., Darrell, T., Keutzer, K., and Suhr, A. Learning adaptive parallel reasoning with language models. *arXiv preprint arXiv:2504.15466*, 2025.
- Ruan, C., Jiang, D., Wang, Y., and Chen, W. Critique-coder: Enhancing coder models by critique reinforcement learning, 2025. URL <https://arxiv.org/abs/2509.22824>.
- Saha, S., Li, X., Ghazvininejad, M., Weston, J., and Wang, T. Learning to plan & reason for evaluation with thinking-llm-as-a-judge, 2025. URL <https://arxiv.org/abs/2501.18099>.
- Sareen, K., Moss, M. M., Sordoni, A., Agarwal, R., and Hosseini, A. Putting the value back in rl: Better test-time scaling by unifying llm reasoners with verifiers, 2025. URL <https://arxiv.org/abs/2505.04842>.
- Setlur, A., Rajaraman, N., Levine, S., and Kumar, A. Scaling test-time compute without verification or rl is suboptimal, 2025. URL <https://arxiv.org/abs/2502.12118>.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Shi, W. and Jin, X. Heimdall: test-time scaling on the generative verification, 2025. URL <https://arxiv.org/abs/2504.10337>.
- Singhi, N., Bansal, H., Hosseini, A., Grover, A., Chang, K.-W., Rohrbach, M., and Rohrbach, A. When to solve, when to verify: Compute-optimal problem solving and generative verification for llm reasoning, 2025. URL <https://arxiv.org/abs/2504.01005>.
- Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Stechly, K., Valmeekam, K., and Kambhampati, S. On the self-verification limitations of large language models on reasoning and planning tasks. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=400v4s3IzY>.
- Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., and Christiano, P. F. Learning to summarize from human feedback. *CoRR*, abs/2009.01325, 2020. URL <https://arxiv.org/abs/2009.01325>.
- Tan, S., Luo, M., Cai, C., Venkat, T., Montgomery, K., Hao, A., Wu, T., Balyan, A., Roongta, M., Wang, C., Li, L. E., Popa, R. A., and Stoica, I. rllm: A framework for post-training language agents, 2025. *Notion Blog*.
- Toshniwal, S., Sorokin, I., Ficek, A., Moshkov, I., and Gitman, I. Genselect: A generative approach to best-of-n. *arXiv preprint arXiv:2507.17797*, 2025.
- Venkatraman, S., Jain, V., Mittal, S., Shah, V., Obando-Ceron, J., Bengio, Y., Bartoldson, B. R., Kailkhura, B., Lajoie, G., Berseth, G., Malkin, N., and Jain, M. Recursive self-aggregation unlocks deep thinking in large language models, 2025. URL <https://arxiv.org/abs/2509.26626>.

- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Wang, Y., Yang, L., Tian, Y., Shen, K., and Wang, M. Co-evolving llm coder and unit tester via reinforcement learning, 2025. URL <https://arxiv.org/abs/2506.03136>.
- Weng, Y., Zhu, M., Xia, F., Li, B., He, S., Liu, S., Sun, B., Liu, K., and Zhao, J. Large language models are better reasoners with self-verification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 2550–2575, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.167. URL <https://aclanthology.org/2023.findings-emnlp.167/>.
- Whitehouse, C., Wang, T., Yu, P., Li, X., Weston, J., Kulikov, I., and Saha, S. J1: Incentivizing thinking in llm-as-a-judge via reinforcement learning, 2025. URL <https://arxiv.org/abs/2505.10320>.
- Wu, Y., Sun, Z., Li, S., Welleck, S., and Yang, Y. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models, 2025. URL <https://arxiv.org/abs/2408.00724>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W.-Y., Zhang, Y.-Q., Yan, L., Qiao, M., Wu, Y., and Wang, M. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Zha, K., Gao, Z., Shen, M., Hong, Z.-W., Boning, D. S., and Katabi, D. Rl tango: Reinforcing generator and verifier together for language reasoning, 2025. URL <https://arxiv.org/abs/2505.15034>.
- Zhang, L., Hosseini, A., Bansal, H., Kazemi, M., Kumar, A., and Agarwal, R. Generative verifiers: Reward modeling as next-token prediction, 2025. URL <https://arxiv.org/abs/2408.15240>.
- Zhao, E., Awasthi, P., and Gollapudi, S. Sample, scrutinize and scale: Effective inference-time search by scaling verification, 2025a. URL <https://arxiv.org/abs/2502.01839>.
- Zhao, W., Aggarwal, P., Saha, S., Celikyilmaz, A., Weston, J., and Kulikov, I. The majority is not always right: Rl training for solution aggregation. *arXiv preprint arXiv:2509.06870*, 2025b.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.
- Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 2024.
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., and Irving, G. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

A. Extended Related Work

Parallel reasoning and test-time scaling. Test-time scaling approaches fall into two paradigms: sequential methods such as long chain-of-thought that iteratively refine a single solution (Madaan et al., 2023), and parallel methods that generate multiple reasoning paths simultaneously (Cobbe et al., 2021; Setlur et al., 2025; Pan et al., 2025; Zhao et al., 2025a; Singhi et al., 2025; Lian et al., 2025). Parallel scaling helps explore diverse solution paths simultaneously; however, it requires effective mechanisms to combine various solutions. For math, majority voting is effective due to objective answers (Wang et al., 2023; Snell et al., 2024; Wu et al., 2025), but this approach is domain-specific and unavailable for domains such as code generation and scientific discovery, where answers are not directly comparable to each other via exact matching. For code generation, prior work (Li et al., 2025a) has explored test-time scaling but requires executable test cases in the evaluation data or generated by larger models like GPT-4 for clustering and selection, while μ Code (Jain et al., 2025) leverages external, multi-turn execution feedback, both of which are specific to code generation. We focus on *self-verification* for parallel reasoners, where a model judges its own generations without external verified or feedback.

Self-verification and self-aggregation Early work by Weng et al. (2023) demonstrated that LLMs can verify their own chain-of-thought reasoning, improving accuracy on arithmetic and commonsense tasks. Related to this, there have been multiple studies analysing the self-refinement capabilities and limitations of LLMs (Madaan et al., 2023; Stechly et al., 2025; Huang et al., 2024), however, they focus on a sequential reasoning setting, while our focus is on explicit self-verification in parallel reasoning. Recently Lu et al. (2025) show that LLMs exhibit a bias toward accepting incorrect solutions during pointwise self-verification. On the other hand, self-aggregation methods like RSA (Venkatraman et al., 2025) and parallel-distill-refine (Madaan et al., 2025) combine solutions generated by the same model, but suffer from diversity collapse (Section 4), resulting in correct solutions being discarded during inference. Such aggregation-based approaches are orthogonal to self-verification and can be combined with better self-verification for more better performance and efficient test-time scaling. Our work addresses limitations of the above prior work by studying pairwise self-verification in a parallel reasoning setup, showing it significantly outperforms pointwise self-verification while preserving solution diversity.

Generative verifiers and reward models. Reward models have been central to LLM alignment and RLHF (Christiano et al., 2017; Ziegler et al., 2019; Stiennon et al., 2020), and have emerged as a natural mechanism for scoring and selecting LLM generations (Cobbe et al., 2021; Zheng et al., 2023). Recent work demonstrates that generative reward models, which produce chain-of-thought reasoning before scoring, substantially outperform discriminative approaches (Mahan et al., 2024; Zhang et al., 2025; Shi & Jin, 2025; Saha et al., 2025), with RL-trained verifiers showing improved judgment performance (Liu et al., 2025c; Whitehouse et al., 2025). Recognizing that pairwise comparison is often easier than absolute scoring, several works have explored pairwise reward models: PairRM (Jiang et al., 2023) trains a discriminative pairwise ranker for ensembling LLM outputs, while GenSelect (Toshniwal et al., 2025) and concurrent work by Mahdavi et al. (2025) scale generative verification for mathematical reasoning. Related to this, Zhao et al. (2025b) propose AggLM, which trains an aggregator via RL to synthesize correct answers for improving majority voting. However, these approaches use *separate* verifiers or aggregator models. Our work differs in two key aspects: (1) we study *self-verification*, where the same parallel reasoner model judges its own outputs. This eliminates the need for additional training with curated judge data, saves memory, and reduces the computational overhead associated with external verification. (2) We co-train the model to be both a solution generator and a pairwise self-verifier in a unified framework.

Co-training for generation and verification. Training separate verifiers incurs significant overhead in compute, memory, and data curation. Recent work has explored co-training a single model for both generation and verification. Sareen et al. (2025) train models to self-verify online during RL, ensuring the verifier sees in-distribution generations and uses this capability to perform better majority voting at test-time for math reasoning. Liu et al. (2025a) extends this to unified CoT verifiers, while Zha et al. (2025) co-trains a separate process reward model that provides intermediate rewards. Offline approaches train verifiers or aggregators on data sampled from the base model (Venkatraman et al., 2025; Ruan et al., 2025). For code generation specifically, several works co-train generators with unit-test producers (Wang et al., 2025; Lee et al., 2025). However, existing co-training methods rely on *pointwise* verification rewards. Our V₁-PairRL framework instead co-trains for *pairwise* self-verification in an online, co-evolving setup, which we show yields superior test-time scaling performance.

B. V₁-PointRL Training Details

To provide a fair comparison for our pairwise verification approach, we train V₁-PointRL using the same co-evolving setup as V₁-PairRL. The key difference lies in the verification objective: instead of comparing pairs of solutions, V₁-PointRL assigns an independent score to each solution.

Verification Reward. For pointwise verification, the model evaluates a single solution s and outputs a confidence score between 1-10 which is normalized to $v \in [0, 1]$. The reward is computed as:

$$r_{\text{verif}} = \mathbb{I}(|v - y| \leq 0.2) \cdot (1 - |v - y|) \quad (5)$$

where $y \in \{0, 1\}$ is the ground truth correctness. The sparsity threshold prevents the “safe bet” collapse mode where the model learns to always output $v = 0.5$ (in practice, not applying the sparsity threshold leads to collapse in solution generation performance and is necessary for stable training).

Training Configuration. All hyperparameters (learning rate, batch size, rollout allocation, etc.) are identical to V₁-PairRL as shown in Table 1. The only difference is the verification prompt and reward computation.

Inference. At inference time, V₁-PointRL generates N solutions and independently scores each one. The solution with the highest pointwise score is selected as the final answer.

C. Hyperparameters

C.1. Inference Sampling Parameters

Inference Framework. We use SGLang (Zheng et al., 2024) for all inference experiments, which provides efficient batched inference for large language models.

Sampling Parameters. We set temperature $T = 0.6$ for all code generation experiments and $T = 1.0$ for all math reasoning experiments to encourage better exploration of the solution space. We use top- p sampling with $p = 0.95$ for all experiments. For experiments with V₁-Infer in Section 4 where we test the inference algorithm on widely used open source LLMs like GPT-OSS-20B, GPT-OSS-120B, Qwen-4B-Instruct-2507, Qwen-4B-Thinking-2507, we set the max generation length to 32768. We find that this does not lead to truncation for most evaluation examples, as also found in Venkatraman et al. (2025). We keep the same generation length for V₁-Infer, pointwise verification as well as RSA.

For trained model evaluations in Section 5 we find that training all models lead to longer chains of thoughts, as is commonly the case in RLVR (DeepSeek-AI, 2025). This leads to frequent truncation, especially for the baseline RL model. To resolve this, we adopt the truncate-and-continue-generation method from prior work (Yang et al., 2025; Sareen et al., 2025). When generation reaches the maximum token budget (`finish_reason=length`), we resume decoding by appending the partially generated assistant output to the message history and issuing a continuation request. If the truncated output does not contain a closing `</thinking>` tag, we append the following transition sentence before continuing: “*Considering the limited time by the user, I have to directly give the required response based on the reasoning till now directly.</thinking>*”. This explicitly closes the thinking block and forces the continuation to produce the final answer. If the truncated output already contains `</thinking>`, we directly continue generation without appending any additional text. We continue the generation for 2K tokens, making the total maximum generation length 34 K for all experiments in Section 5.

Number of Samples. We generate $N \in \{8, 16\}$ candidate solutions for most experiments. For budget-matched comparisons between pointwise and pairwise verification, we additionally evaluate pointwise verification with $N = 32$ samples to match the total LLM calls of pairwise verification with $N = 16$ and budget $3\times$. This budget-matched comparison is performed only for GPT-OSS-20B and Qwen3-4B-Instruct models.

Verification Budget. For V₁-Infer experiments in Section 4, we evaluate verification budgets of $1\times$, $2\times$, and $3\times$ the number of base solutions (N). Full results across all budgets are provided in the appendix figures.

Experimental Runs. All inference experiments for base models (Section 4) are run once. All trained model results (Section 5) are run 3 times with different random seeds, and we report the mean. Due to the computational cost of running multiple seeds, we limit the budget exploration for trained models to $1\times$ and $2\times$, though we believe performance can further scale with additional budget for the pairwise method (V₁-Infer).

Swiss Refinement Algorithm Parameters. For V₁-Infer, we use the following settings: Minimum degree $d_{\min} = 2$ (each solution compared at least twice in coverage phase), Swiss window size $h = 8$, Confidence floor set to a small value, $\tau = 0.1$ for weighted aggregation. We do not tune parameters to any specific benchmark. We use the same parameters across benchmarks or models across all experiments.

C.2. Training Hyperparameters

Table 1 summarizes the key training hyperparameters. We train models using rLLM (Tan et al., 2025) and verl (Sheng et al., 2024) backend.

Table 1. Training hyperparameters for V₁.

Hyperparameter	Qwen3-4B-Inst
Learning rate	1×10^{-6}
Batch size	64
Rollouts per prompt (G)	4
Judge rollouts	4
Max prompt length	10240
Max response length	24576
Temperature	0.6
Top-p	0.95
Clip ratio (low)	0.2
Clip ratio (high)	0.28
λ (loss weight)	1.0
KL coefficient	0
Entropy coefficient	0
Std normalization	No
Loss aggregation	token-mean

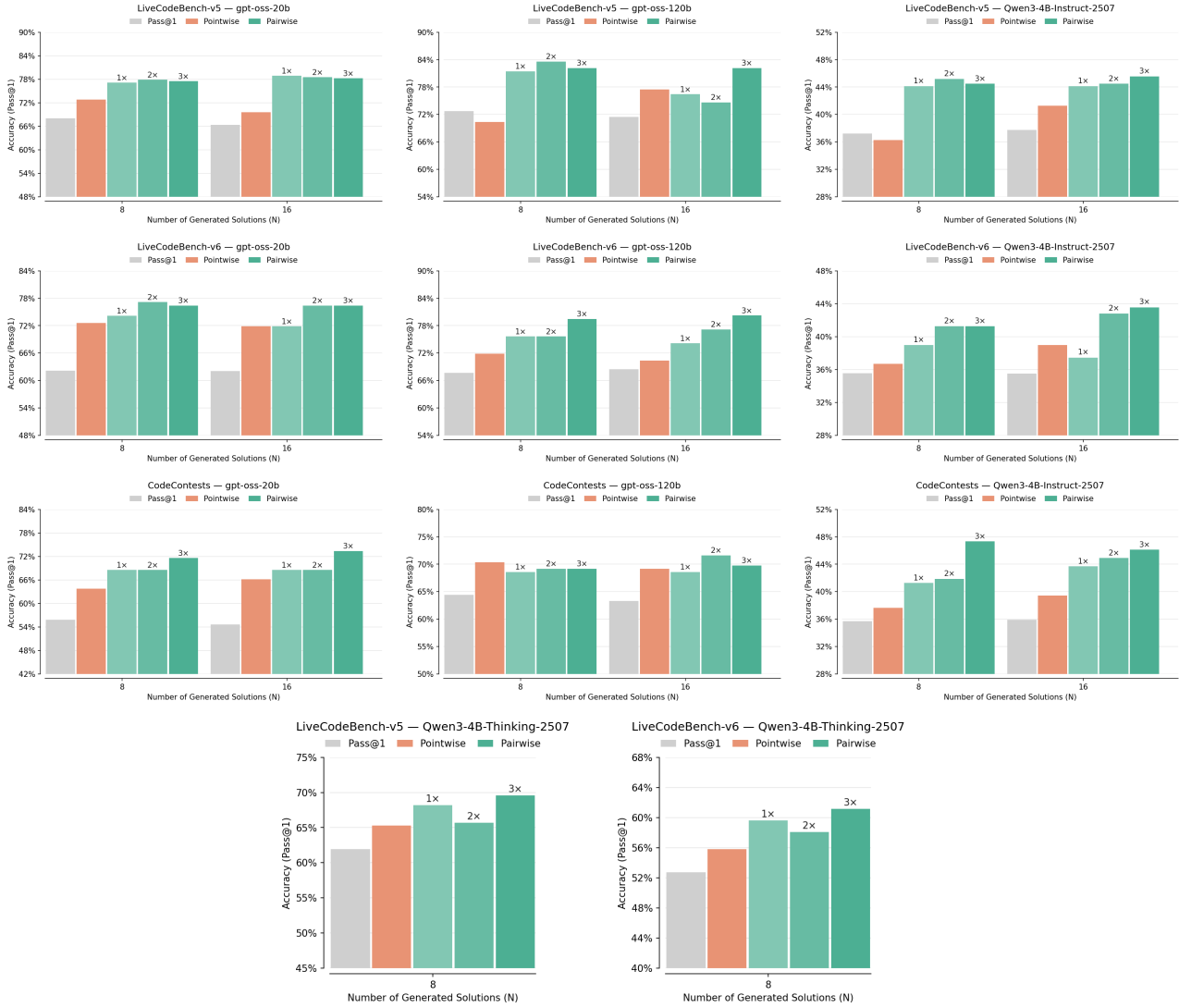


Figure 7. Self-verification bar plots across benchmarks and models (LiveCodeBench-v5, LiveCodeBench-v6, and CodeContests). Each plot compares Pass@1, pointwise verification, and pairwise verification (budgets $1 \times / 2 \times / 3 \times$) at $N \in \{8, 16\}$. For Qwen3-4B-Thinking (bottom row), we run experiments only on LiveCodeBench at $N = 8$ to manage compute.

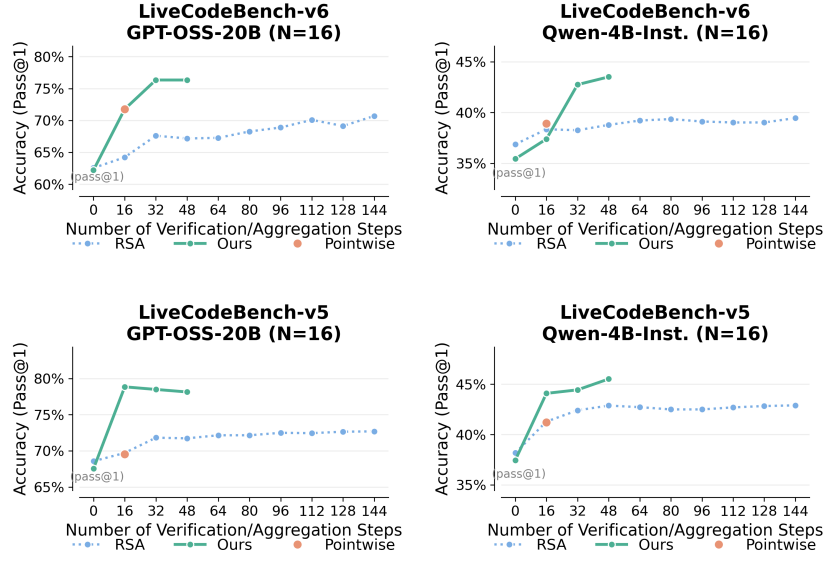


Figure 8. Comparing Pairwise Verification with RSA. Top row: LiveCodeBench-v6. Bottom row: LiveCodeBench-v5. Columns (left→right): gpt-oss-20b, Qwen3-4B-Instruct-2507. Here number of generations by the base model is 16, followed by verification (pointwise, pairwise) or aggregation (RSA).

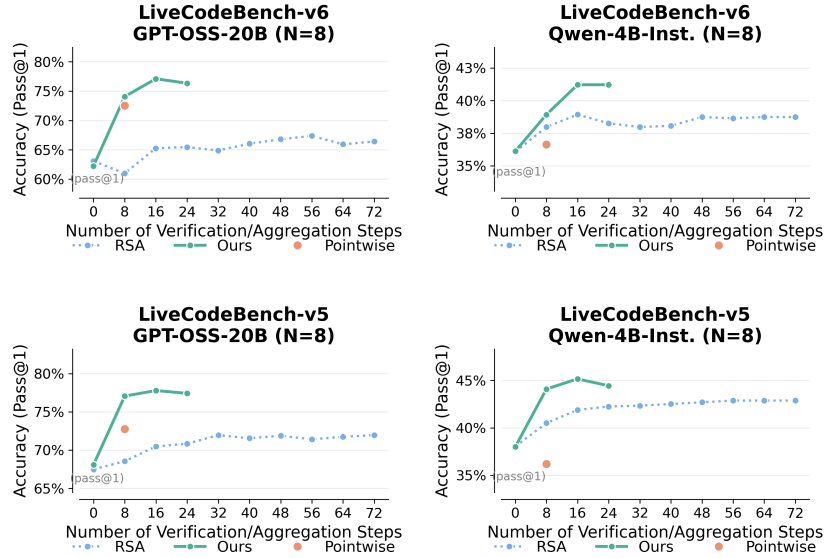


Figure 9. Comparing Pairwise Verification with RSA. Top row: LiveCodeBench-v6. Bottom row: LiveCodeBench-v5. Columns (left→right): gpt-oss-20b, Qwen3-4B-Instruct-2507. Here number of generations by the base model is 8, followed by verification (pointwise, pairwise) or aggregation (RSA).

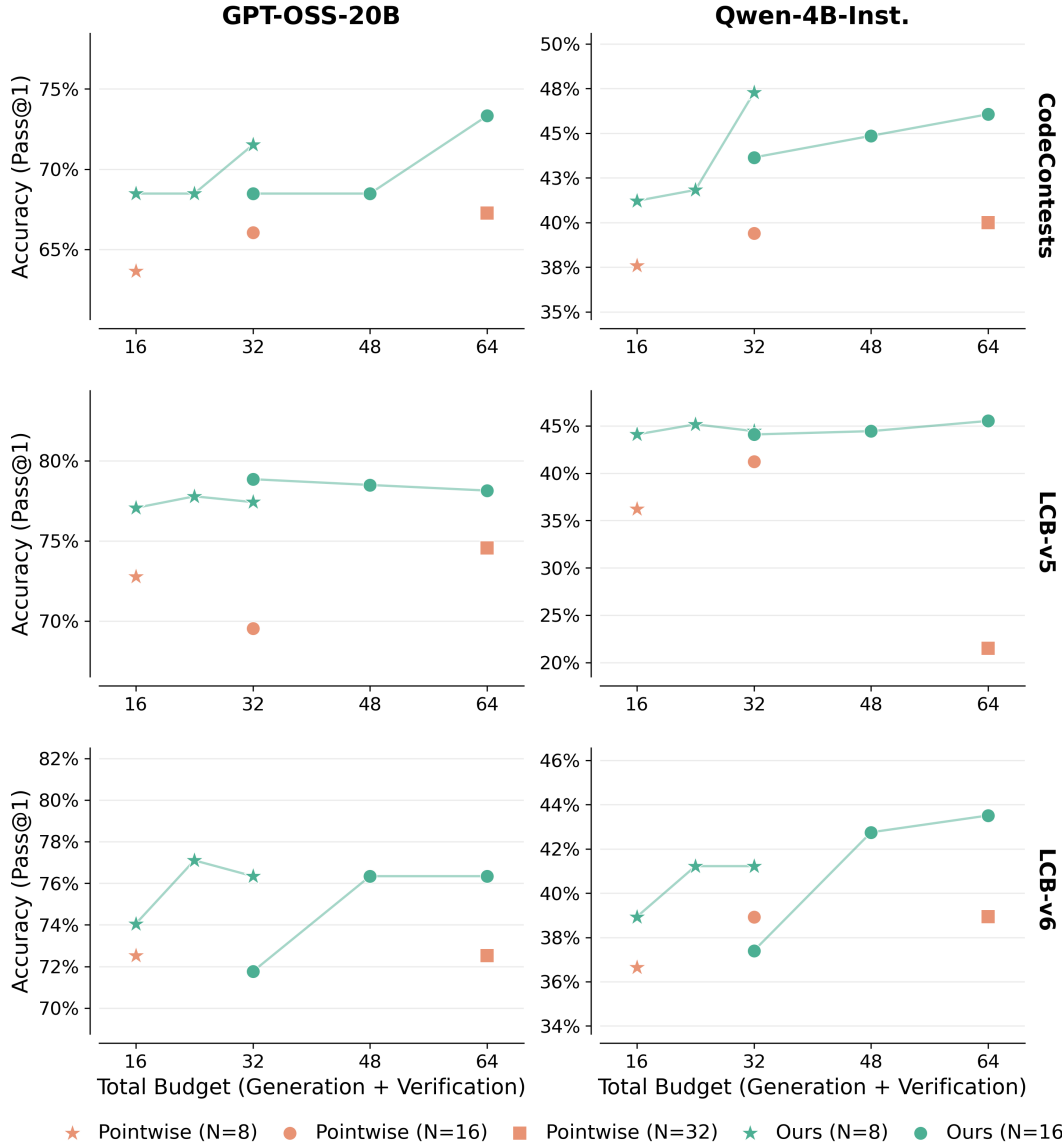


Figure 10. **Budget vs accuracy per benchmark.** Accuracy vs. total budget (generation + verification calls) for CodeContests (top row), LiveCodeBench-v5 (middle row), and LiveCodeBench-v6 (bottom row). Left column: GPT-OSS-20B. Right column: Qwen-4B-Inst. Stars denote $N = 8$ base generations, circles denote $N = 16$, squares denote $N = 32$ (pointwise only).

D. Detailed Algorithm

This section provides the full specification of the helper procedures used in Algorithm 1. The main paper presents a high-level view for clarity; here we give the exact update rules and pairing strategies required for reproducibility.

State. We maintain a state $\mathcal{T} = (\{\mu_i\}_{i=1}^N, \{d_i\}_{i=1}^N, \mathcal{H})$, where μ_i is the current estimated quality score of solution s_i , d_i is the number of distinct opponents it has been compared against, and $\mathcal{H} \subset [N] \times [N]$ records previously compared pairs.

CoveragePairs. Ensures minimum degree while anchoring comparisons to similar-quality peers.

Algorithm 2 COVERAGEPAIRS($\mathcal{T}, d_{\min}$)

Require: State $\mathcal{T} = (\{\mu_i\}, \{d_i\}, \mathcal{H})$, minimum degree $d_{\min}$

Ensure: Disjoint pair set $\mathcal{P}$

```

1:  $\mathcal{P} \leftarrow \emptyset, \mathcal{U} \leftarrow \emptyset$  ▷ used indices
2:  $\mathcal{L} \leftarrow \{i : d_i < d_{\min}\}$ ; sort  $\mathcal{L}$  by increasing  $d_i$ 
3: for each  $i \in \mathcal{L}$  do
4:   if  $i \in \mathcal{U}$  then
5:     continue
6:   end if
7:    $\mathcal{C} \leftarrow \{j \neq i : j \notin \mathcal{U}, (i, j) \notin \mathcal{H}\}$ 
8:   if  $\mathcal{C} = \emptyset$  then
9:     continue
10:  end if
11:   $j \leftarrow \arg \min_{j \in \mathcal{C}} |\mu_i - \mu_j|$ 
12:   $\mathcal{P} \leftarrow \mathcal{P} \cup \{(i, j)\}$ 
13:   $\mathcal{U} \leftarrow \mathcal{U} \cup \{i, j\}$ 
14: end for
15: return  $\mathcal{P}$ 

```

UpdateStats. This is where the mathematical aggregation is implemented. Note. In practice, the update of μ_i is

Algorithm 3 UPDATESTATS($\mathcal{T}, \mathcal{O}, \tau$)

Require: State $\mathcal{T} = (\{\mu_i\}, \{d_i\}, \mathcal{H})$, outcomes $\mathcal{O}$, floor $\tau > 0$

Ensure: Updated state $\mathcal{T}$

```

1: for each  $(i, j, w, r_i, r_j) \in \mathcal{O}$  do
2:    $v_{ij} \leftarrow \mathbb{I}[w = \text{A}] + 0.5 \cdot \mathbb{I}[w = \text{tie}]$ 
3:    $w_{ij} \leftarrow \max\left(\frac{|r_i - r_j|}{9}, \tau\right)$ 
4:   update  $\mu_i, \mu_j$  using Eq. (1)
5:   if  $(i, j) \notin \mathcal{H}$  then
6:      $d_i \leftarrow d_i + 1; d_j \leftarrow d_j + 1$ 
7:      $\mathcal{H} \leftarrow \mathcal{H} \cup \{(i, j), (j, i)\}$ 
8:   end if
9: end for

```

implemented incrementally using sufficient statistics, but Eq. (1) fully specifies the aggregation semantics.

The above procedures jointly ensure (i) early stabilization of the comparison topology via minimum-degree coverage and (ii) efficient use of remaining budget through uncertainty-focused Swiss refinement, enabling accurate ranking with $O(N)$ -scale pairwise verification.

SwissPairs. Allocates remaining budget to the most informative near-ties.

Algorithm 4 SWISSPAIRS($\pi, \mathcal{T}, h$)

Require: Ranking π , state $\mathcal{T} = (\{\mu_i\}, \{d_i\}, \mathcal{H})$, window size h

Ensure: Disjoint pair set $\mathcal{P}$

```

1:  $\mathcal{P} \leftarrow \emptyset, \mathcal{U} \leftarrow \emptyset$ 
2: for  $k = 1$  to  $N$  do
3:    $i \leftarrow \pi_k$ 
4:   if  $i \in \mathcal{U}$  then
5:     continue
6:   end if
7:    $\mathcal{C} \leftarrow \emptyset$ 
8:   for  $t = k + 1$  to  $\min(k + h, N)$  do
9:      $j \leftarrow \pi_t$ 
10:    if  $j \in \mathcal{U}$  then
11:      continue
12:    end if
13:    add  $(|\mu_i - \mu_j|, \mathbb{I}[(i, j) \in \mathcal{H}], j)$  to  $\mathcal{C}$ 
14:  end for
15:  if  $\mathcal{C} \neq \emptyset$  then
16:    choose  $j$  with lexicographically minimal key in  $\mathcal{C}$ 
17:     $\mathcal{P} \leftarrow \mathcal{P} \cup \{(i, j)\}$ 
18:     $\mathcal{U} \leftarrow \mathcal{U} \cup \{i, j\}$ 
19:  end if
20: end for
21: return  $\mathcal{P}$ 

```

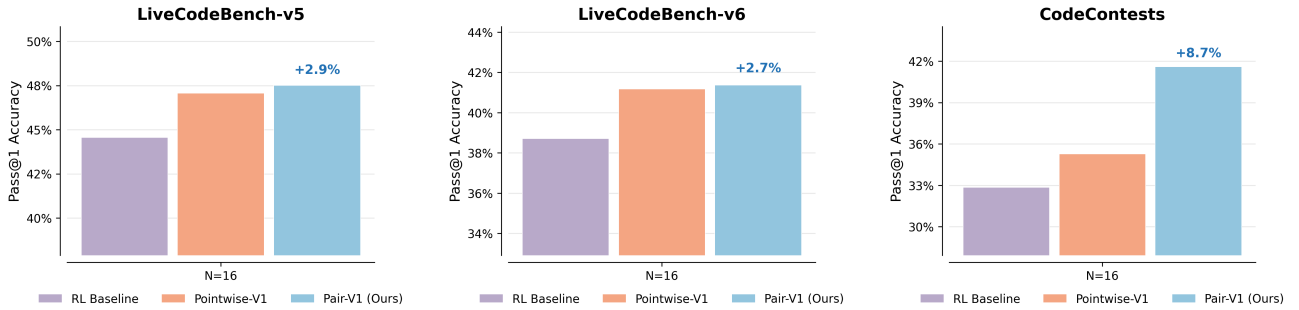


Figure 11. V₁-PairRL vs RL baseline across benchmarks (N=16). Left: LiveCodeBench-v5. Middle: LiveCodeBench-v6. Right: CodeContests. Co-training with pairwise verification consistently improves Pass@1 accuracy across all benchmarks.

E. Verification Prompts

This section provides the exact prompts used for pointwise and pairwise self-verification in our experiments. Both verification methods use a 1–10 rating scale for fair comparison.

E.1. Pointwise Verification Prompts

E.1.1. CODE GENERATION

Pointwise Code Verification Prompt

You are an expert code reviewer. Rate the correctness of a solution to a programming problem.

****Evaluation Guidelines:****

- Analyze the problem’s requirements and constraints.
- Mentally trace the solution with test cases (including edge cases) to verify correctness.
- Give a higher score if the solution is robust and fault-tolerant.

****Problem****
{problem}

****Solution****
{code}

****Output Format:****
First, provide your step-by-step reasoning. Then, on a new line, provide your final rating using the EXACT tags below. Add no other text after the tags.
<rating>INTEGER_1_TO_10</rating>

****Rating Rules:****

- Rate correctness on a 1-10 scale (10 = correct & robust, 5 = borderline, 1 = incorrect).

Please provide your analysis now.

E.1.2. MATH REASONING

Pointwise Math Verification Prompt

You are an expert math contest grader. Rate the correctness of a submission based solely on the final answer.

****Evaluation Guidelines:****

- Extract the submission’s final answer. Use any provided reasoning only to help you assess whether the stated final answer is trustworthy. Do not award credit for method quality or rigor.
- Carefully analyze the problem statement and the submission to assess whether the final answer is correct. Grade only the final answer.

****Problem****
{problem}

****Solution****
{solution}

****Output Format:****
First, provide your reasoning (what checks you performed). Then, on a new line, provide your final rating using the EXACT tag below. Add no other text after the tag.
<rating>INTEGER_1_TO_10</rating>

****Rating Rules:****

- Rate correctness on a 1-10 scale (10 = certainly correct, 8 = very likely correct, 5 = uncertain/borderline, 3 = likely incorrect, 1 = certainly incorrect).

Please provide your analysis now.

E.2. Pairwise Verification Prompts

E.2.1. CODE GENERATION

Pairwise Code Verification Prompt

You are an expert code reviewer. Compare two solutions to a programming problem and rate their correctness.

****Evaluation Guidelines:****

- Analyze the problem's requirements and constraints.
- Mentally trace each solution with test cases (including edge cases) to verify correctness.
- If both solutions appear correct, prefer the more robust and fault-tolerant one.

****Problem****

{problem}

****Solution A****

{code_A}

****Solution B****

{code_B}

****Output Format:****

First, provide your step-by-step reasoning. Then, on separate new lines, provide your final ratings using the EXACT tags below. Add no other text after the tags.

<rating_A>INTEGER.1.TO.10</rating_A>

<rating_B>INTEGER.1.TO.10</rating_B>

****Rating Rules:****

- Rate correctness on a 1-10 scale (10 = correct & robust, 5 = borderline, 1 = incorrect).
 - The higher rating wins. Equal ratings imply a tie.
- Please provide your analysis now.

E.2.2. MATH REASONING

Pairwise Math Verification Prompt

You are an expert math contest grader. Compare two submissions and rate correctness based solely on the final answer.

****Evaluation Guidelines:****

- Extract each submission's final answer. Use any provided reasoning only to help you assess whether the stated final answer is trustworthy. Do not award credit for method quality or rigor.
- Carefully analyze the problem statement and the submissions to assess whether each final answer is correct. Grade only the final answer.

****Problem****

{problem}

****Solution A****

{sol_A}

****Solution B****

{sol_B}

****Output Format:****

First, provide your reasoning (what checks you performed). Then, on separate new lines, give ratings using the EXACT tags below. Add no other text after the tags.

<rating_A>INTEGER.1.TO.10</rating_A>

<rating_B>INTEGER.1.TO.10</rating_B>

****Rating Rules:****

- Rate correctness on a 1-10 scale (10 = certainly correct, 8 = very likely correct, 5 = uncertain/borderline, 3 = likely incorrect, 1 = certainly incorrect).
- Higher rating wins. Equal ratings imply a tie.

Please provide your analysis now.